Design Patterns, Principles, and Architectures using .NET

New Batch: March 4, 2026, 6:30 AM – 8:00 AM, IST
Duration: 2 Months (Monday to Friday)
Fees: 5000 (INR) or 65 USD
Registration Link: https://forms.gle/vPceecJ5HpPUfrK3A
Telegram Group: https://telegram.me/trainingcsharponline
WhatsApp Group: https://chat.whatsapp.com/JcYveKZsw0c89VlsoUb162
Course Details: https://dotnettutorials.net/lesson/design-patterns-online-training/
Phone/WhatsApp Number: +91 7021801173
Email ID: info@dotnettutorials.net

Demo Sessions: 9, 10, and 11 March, 6:30 AM to 8 AM

Demo Zoom Credentials:
Link: https://us06web.zoom.us/j/88072310817?pwd=Uln7kEwDvBQt2wPXnjuV5bJbXyZatB.1
Meeting ID: 880 7231 0817
Passcode: 580718

Module 1: Foundations for Design Patterns (Core Concepts)

In this module, you will learn the essential foundation required to understand and apply design patterns confidently. You will refresh core OOP concepts, learn how SOLID principles improve code maintainability, and understand how Dependency Injection and IoC help in building loosely coupled applications. You will also explore how MVC and Options patterns fit into real .NET applications, and you will strengthen your development workflow using unit testing and mocking so you can validate pattern-based designs effectively.

Chapter 1: Introduction to Design Patterns

This chapter introduces design patterns as reusable solutions to common software design problems. You will understand why poor design leads to rigid and hard-to-maintain applications and how design patterns help build flexible, scalable, and clean .NET applications. The chapter also explains when design patterns should be used and when they should be avoided to prevent overengineering.

- What are Design Patterns?
- History of Design Patterns (Gang of Four)
- Why Design Patterns are needed in real-world applications
- Problems caused by bad design
- Benefits of using Design Patterns
- Design Patterns vs Algorithms vs Frameworks
- When to use and when NOT to use Design Patterns
- Real-world examples of Design Patterns in .NET
- Understanding overengineering and misuse of patterns

Chapter 2: Object-Oriented Programming Concepts (OOP)

This chapter revises core OOP concepts that form the foundation of all design patterns. You will learn how proper object-oriented design helps achieve loose coupling, better reusability, and easier maintenance, which are essential for applying design patterns correctly.

- What is Object-Oriented Programming?
- Core OOP principles: Encapsulation, Abstraction, Inheritance, Polymorphism
- Object relationships: Association, Aggregation, Composition
- Interface vs Abstract Class
- Composition over inheritance
- Tight coupling vs Loose coupling
- How OOP enables Design Patterns

Chapter 3: SOLID Design Principles

This chapter explains the SOLID principles that guide clean and maintainable object-oriented design. Students will learn how following SOLID principles reduces code complexity, improves flexibility, and naturally leads to better use of design patterns in real-world applications.

- Introduction to SOLID Principles
- Single Responsibility Principle (SRP)
- Open/Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)
- Common SOLID violations and refactoring ideas
- How SOLID principles enable Design Patterns

Chapter 4: Dependency Injection (DI) & Inversion of Control (IoC)

This chapter focuses on reducing tight coupling using Dependency Injection and Inversion of Control. Students will understand why creating dependencies directly is problematic and how DI improves testability, flexibility, and maintainability in modern .NET applications.

- What is Inversion of Control?
- Problems with tightly coupled code
- What is Dependency Injection?
- Types of Dependency Injection (Constructor, Property, Method)
- DI lifetimes: Transient, Scoped, Singleton
- Built-in Dependency Injection in .NET
- Service registration and resolution
- DI best practices

Chapter 5: MVC Pattern

This chapter explains the **MVC (Model–View–Controller) pattern** as a core architectural pattern used heavily in ASP.NET Core applications. Students will learn how MVC separates concerns so UI, business logic, and request handling don't get mixed together, making applications easier to maintain, test, and extend.

- What MVC is and why it exists (separation of concerns)
- Responsibilities of **Model**, **View**, and **Controller**
- Request flow in ASP.NET Core MVC (routing → controller → view/model)
- Fat controller vs thin controller (best practices)
- Where business logic should live (services, domain layer)
- MVC vs Web API (when to use which)
- Real-world mapping: pages, forms, validations, and action results

Chapter 6: Options Pattern

This chapter teaches how to read appsettings.json (and other configuration sources) using strongly-typed classes in ASP.NET Core. Instead of scattering configuration keys across the codebase, students learn to bind configuration to typed options, inject them through DI, and validate them cleanly for real-world applications.

- What Options Pattern is and why it matters in real projects
- Creating options classes (e.g., JwtOptions, EmailOptions, CacheOptions)
- Binding configuration sections to options
- IOptions<T> vs IOptionsSnapshot<T> vs IOptionsMonitor<T> (when to use what)
- Named options (multiple configurations for the same options type)
- Options validation (Data Annotations + custom validation)
- Real-time example: JWT/Auth settings or Email settings via options

Chapter 7: Unit Testing in .NET (xUnit/NUnit + Mocking)

This chapter introduces unit testing in .NET so students can confidently validate business logic and design decisions. It focuses on writing clean tests, mocking dependencies, testing async code, and building testable services—skills that directly improve how design patterns are applied in real projects.

- Unit tests vs integration tests vs end-to-end tests
- AAA pattern (Arrange–Act–Assert) and test naming conventions
- Testing services that use DI (mocking dependencies)
- Mocking fundamentals (Moq/NSubstitute concepts)
- Testing async methods and exception scenarios
- What to test and what not to test (avoid testing implementation details)
- Improving design using tests (testability as a design driver)

Module 2: GoF Overview + How to Choose the Right Pattern

In this module, you will understand what the Gang of Four (GoF) design patterns are, why they are categorized into Creational, Structural, and Behavioral patterns, and how to identify which category a real problem belongs to. You will also learn the basic UML ideas needed to read and understand pattern diagrams, and you will learn a practical approach to selecting the right pattern based on the problem.

Chapter 8: GoF Design Patterns Overview

This chapter gives a big-picture view of the GoF patterns and how they are grouped into creational, structural, and behavioural categories. You will also learn basic UML ideas and a simple approach to choosing the right pattern for a given problem.

- Gang of Four (GoF) Design Patterns overview
- Classification of GoF patterns
  - Creational Patterns
  - Structural Patterns
  - Behavioural Patterns
- Pattern relationships and combinations
- UML basics used in Design Patterns
- How to select the right pattern for a problem

Module 3: Creational Design Patterns (Object Creation Mastery)

In this module, you will learn patterns that focus on creating objects in a controlled, reusable, and flexible way. You will understand how to avoid tight coupling caused by direct object creation, and you will learn how to design object creation that supports scalability and change. By the end, you will be able to implement common creational patterns like Singleton, Factory Method, Abstract Factory, Builder, and Prototype in .NET applications with clear real-world use cases.

Chapter 9: Singleton Pattern

This chapter explains how to ensure only one instance of a class exists and when that is useful in real applications. You will also learn thread safety, lazy initialization, common problems, and why DI-managed singletons are often a safer choice.

- Intent and motivation
- Real-world use cases
- Basic Singleton implementation
- Thread-safe Singleton
- Lazy vs Eager initialization
- Singleton pitfalls
- Singleton vs Dependency Injection
- When to avoid Singleton

Chapter 10: Factory Method Pattern

This chapter shows how Factory Method helps create objects without hardcoding concrete classes everywhere. You will learn how it supports the Open/Closed Principle and makes your code easier to extend when new types are added.

- Intent and motivation
- Factory Method vs Simple Factory
- Encapsulating object creation
- Open/Closed Principle with Factory Method
- Real-world examples in C#
- Advantages and limitations

Chapter 11: Abstract Factory Pattern

This chapter explains how Abstract Factory creates families of related objects without depending on their concrete implementations. You will understand how it prevents mixing incompatible objects and is useful for switching between multiple providers or environments.

- Intent and motivation
- Factory of factories concept
- Creating families of related objects
- Abstract Factory vs Factory Method
- Use cases in UI and database providers
- Advantages and drawbacks

Chapter 12: Builder Pattern

This chapter teaches how Builder helps construct complex objects step by step, especially when many optional values are involved. You will learn how fluent builders improve readability and how builder works well with immutable objects.

- Intent and motivation
- Step-by-step object construction
- Immutable objects with Builder
- Fluent Builder pattern
- Builder vs Factory
- Real-world C# examples

Chapter 13: Prototype Pattern

This chapter explains how Prototype creates new objects by cloning existing ones instead of building from scratch. You will understand shallow vs deep copying and when cloning is a practical solution in C# applications.

- Intent and motivation
- Object cloning concepts
- Shallow copy vs Deep copy
- Implementing Prototype in C#
- Use cases and limitations

Module 4: Structural Design Patterns (Composition + Integration)

In this module, you will learn how to organize classes and objects to build larger systems in a clean, extensible, and maintainable way. You will understand how to connect incompatible interfaces, simplify complex subsystems, add responsibilities without changing existing code, and optimize object usage when memory matters. By the end, you will confidently apply patterns like Adapter, Bridge, Composite, Decorator, Facade, Flyweight, and Proxy to improve the structure of real .NET applications.

Chapter 14: Adapter Pattern

This chapter shows how Adapter helps you connect incompatible interfaces, especially when working with legacy code or third-party libraries. You will learn how adapters allow integration without changing your core application design.

- Intent and motivation
- Object Adapter vs Class Adapter
- Integrating legacy systems
- Interface compatibility
- Real-world examples

Chapter 15: Bridge Pattern

This chapter explains how Bridge separates abstraction from implementation so both can change independently. You will see how it avoids too many subclasses when your system has multiple variations.

- Intent and motivation
- Decoupling abstraction from implementation
- Bridge vs Strategy
- When Bridge pattern is useful

Chapter 16: Composite Pattern

This chapter teaches how Composite helps model tree structures so you can treat single objects and groups of objects in the same way. You will see real examples like menus, folders, and category hierarchies.

- Intent and motivation
- Tree structures and hierarchies
- Leaf and Composite object
- Real-world examples (menus, file systems)

Chapter 17: Decorator Pattern

This chapter explains how Decorator adds extra behaviour to an object without changing its class. You will learn practical uses like adding logging, caching, validation, or retry behaviour in a clean and flexible way.

- Intent and motivation
- Extending behaviour dynamically
- Decorator vs Inheritance
- Real-world scenarios (logging, caching)

Chapter 18: Facade Pattern

This chapter shows how Facade provides a simple entry point to a complex subsystem. You will learn how it hides internal complexity and makes modules easier for other developers to use.

- Intent and motivation
- Simplifying complex subsystems
- Facade vs Adapter
- Use cases in layered architectures

Chapter 19: Flyweight Pattern

This chapter explains how Flyweight reduces memory usage when many similar objects are created. You will understand intrinsic vs extrinsic state and when this optimization is actually worth using.

- Intent and motivation
- Memory optimization
- Intrinsic vs Extrinsic state
- Performance considerations

Chapter 20: Proxy Pattern

This chapter teaches how Proxy controls access to another object by acting as a wrapper. You will learn common types like virtual, protection, and remote proxies, and how they help with lazy loading and security.

- Intent and motivation
- Types of proxies
  - Virtual Proxy
  - Protection Proxy
  - Remote Proxy
- Real-world examples (lazy loading, security)

Module 5: Behavioural Design Patterns (Communication)

In this module, you will learn patterns that focus on communication, responsibilities, and flow of control between objects. You will understand how to design flexible workflows, implement clean event-driven interactions, and remove complex conditional logic by shifting behavior into well-structured objects. By the end, you will be able to apply patterns like Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Interpreter, Template Method, and Visitor to solve real project challenges in a clean and testable way.

Chapter 21: Chain of Responsibility Pattern

This chapter explains how requests can pass through a pipeline of handlers until one handles them. You will relate it to ASP.NET Core middleware and see how it makes validation and processing logic easier to extend.

- Intent and motivation
- Request handling pipeline
- Middleware analogy
- Real-world use cases

Chapter 22: Command Pattern

This chapter shows how Command turns actions into objects so they can be queued, logged, or undone. You will understand how it separates the requester from the executor and supports workflows like undo/redo.

- Intent and motivation
- Encapsulating requests
- Undo/Redo functionality
- Command vs Strategy

Chapter 23: Iterator Pattern

This chapter explains how Iterator allows safe traversal of collections without exposing their internal structure. You will learn how it relates to foreach and when custom iterators are useful.

- Intent and motivation
- Traversing collections
- Custom iterators
- Iterator vs foreach

Chapter 24: Mediator Pattern

This chapter teaches how Mediator reduces direct communication between many objects by introducing a central coordinator. You will see how this simplifies complex interactions and improves maintainability.

- Intent and motivation
- Reducing object coupling
- Centralized communication
- Real-world scenarios

Chapter 25: Memento Pattern

This chapter explains how Memento captures an object's state so it can be restored later without breaking encapsulation. You will see how it supports undo features and safe rollback scenarios.

- Intent and motivation
- Capturing object state
- Undo mechanisms
- Encapsulation preservation

Chapter 26: Observer Pattern

This chapter shows how Observer lets objects subscribe and react when something changes. You will connect it to .NET events and delegates and understand how it differs from pub-sub messaging.

- Intent and motivation
- Event-driven programming
- .NET events and delegates
- Observer vs Pub-Sub

Chapter 27: State Pattern

This chapter explains how State changes an object's behaviour based on its current state without large if-else blocks. You will see how it simplifies workflows like order processing or approval systems.

- Intent and motivation
- State transitions
- Eliminating conditional logic
- Real-world examples

Chapter 28: Strategy Pattern

This chapter teaches how Strategy allows switching between different algorithms at runtime. You will see how it is useful for business rules like pricing, discounts, and routing decisions.

- Intent and motivation
- Dynamic algorithm selection
- Strategy vs State
- Use cases in business rules

Chapter 29: Interpreter Pattern

This chapter introduces Interpreter for evaluating expressions based on a simple grammar. You will learn where it is useful for rule evaluation and why it becomes difficult for complex languages.

- Intent and motivation
- Expression evaluation
- Grammar representation
- Practical limitations

Chapter 30: Template Method Pattern

This chapter explains how Template Method defines a fixed workflow while allowing certain steps to be customized. You will understand how it helps reuse common process logic in a controlled way.

- Intent and motivation
- Defining algorithm skeletons
- Hook methods
- Template vs Strategy

Chapter 31: Visitor Pattern

This chapter teaches how Visitor adds new operations to an existing object structure without modifying the objects. You will understand the idea of double dispatch and why Visitor can be powerful but complex.

- Intent and motivation
- Adding operations without modification
- Double dispatch concept
- Use cases and drawbacks

Module 6: Non-GoF Patterns for Real Projects

In this module, you will learn modern enterprise patterns that are heavily used in real-world .NET projects, especially in layered architecture applications. You will understand how Repository and Unit of Work simplify data access and transaction consistency, how CQRS separates read and write responsibilities for better scalability, and how the Result Pattern helps us handle success and failure scenarios cleanly without spreading exception-driven logic everywhere.

Chapter 32: Repository, Generic Repository & Unit of Work

This chapter explains how Repository separates business logic from data access so code becomes cleaner and testable. You will also learn how Generic Repository and Unit of Work handle common CRUD and transaction scenarios, along with best practices to avoid misuse.

- What is Repository Pattern?
- Purpose and benefits
- Generic Repository Pattern
- Limitations of Generic Repository
- Unit of Work Pattern
- Transaction management
- Repository + Unit of Work with EF Core
- Best practices and anti-patterns

Chapter 33: CQRS (Command Query Responsibility Segregation)

This chapter explains how CQRS separates write operations from read operations to make systems clearer and easier to scale. You will learn commands vs queries, read vs write models, and when CQRS is unnecessary for smaller applications.

- What is CQRS?
- Commands vs Queries
- Read model vs Write model
- Benefits of CQRS
- CQRS with and without MediatR
- When CQRS should NOT be used

Chapter 34: Result Pattern (Success/Failure Without Exceptions)

This chapter explains how to return success/failure outcomes from services and CQRS handlers using a consistent Result / Result<T> approach. Students learn how to avoid exception-driven control flow for expected failures and how to produce clean, predictable API responses with structured error messages.

- Why Result Pattern is needed (clean success/failure flow)
- Result vs Result<T> and what fields to include (message, error code, errors list)
- Validation errors using Result Pattern (without exceptions)
- Mapping Result to HTTP responses in Web API (200/400/404/409 patterns)
- Chaining results and early returns for clean service logic
- Using Result Pattern inside CQRS command handlers
- Best practices for consistent error handling across microservices

Module 7: Modern Distributed Systems Patterns (Microservices)

In this module, you will learn patterns that help you build reliable and scalable systems, especially in microservices and distributed environments. You will understand event-driven communication, how to handle failures using retry and circuit breaker strategies, and how to protect systems using rate limiting and throttling. By the end, you will know how these patterns improve resilience, stability, and performance in real production systems.

Chapter 35: Event-Driven Design Patterns

This chapter introduces event-driven architecture where systems communicate using events rather than direct calls. You will learn publish-subscribe, event notification, event-carried state transfer, and how patterns like Saga help manage long-running workflows.

- What is Event-Driven Architecture?
- Events vs Commands vs Messages
- Publish-Subscribe Pattern
- Event Notification Pattern
- Event-Carried State Transfer
- Saga Pattern (intro)
- Real-world event-driven systems

Chapter 36: Retry / Circuit Breaker Patterns (Resilience Patterns)

This chapter teaches **resilience patterns** that make services stable under failures like timeouts, transient network errors, and downstream outages. Students will learn how **Retry** and **Circuit Breaker** help keep systems responsive and prevent cascading failures in microservices-based .NET applications.

- What “transient failures” are and why resilience matters
- **Retry pattern**: when retry helps and when it makes things worse
- Exponential backoff and jitter (conceptual understanding)
- **Circuit Breaker pattern**: closed/open/half-open states
- Preventing cascading failures in microservices
- Timeouts + retries: the correct pairing
- Observability: logging/metrics around retries and breaker trips
- Real-world usage in .NET (concept mapping to Polly-style policies)

Chapter 37: Rate Limiting / Throttling Pattern

This chapter explains how **Rate Limiting / Throttling** protects APIs from abuse and traffic spikes by controlling how many requests a client can make in a time window. Students will learn common strategies and where to apply them (API Gateway, reverse proxy, or service-level) in real .NET systems.

- Rate limiting vs throttling (what each means in practice)
- Why it’s needed (DDoS-ish bursts, bot traffic, noisy tenants, cost control)
- Common strategies: fixed window, sliding window, token bucket, leaky bucket (high level)
- Per-IP, per-user, per-API-key, per-tenant limits
- Handling limits gracefully (HTTP 429, retry-after idea)
- Where to implement: API Gateway vs per service
- Rate limiting + authentication/authorization flow (concept)
- Real-world use cases: login endpoints, OTP, search APIs, public APIs

Module 8: Architecture Patterns (Structuring Enterprise Apps)

In this module, you will learn how to structure large applications using proven architecture approaches used in enterprise .NET systems. You will understand how Clean Architecture and Onion Architecture organize code into layers with clear dependency rules, and how Domain-Driven Design (DDD) helps you model complex business domains using entities, value

objects, aggregates, and domain services. By the end, you will be able to design applications that are scalable, testable, and easy to evolve over time.

Chapter 38: Clean Architecture

This chapter teaches how to structure an application so the domain and business rules stay independent of frameworks and databases. You will learn the dependency rule, layers, and how Clean Architecture fits naturally with ASP.NET Core projects.

- What is Clean Architecture?
- Uncle Bob's Dependency Rule
- Clean Architecture layers
- Domain-centric design
- Role of DTOs and use cases
- Clean Architecture with ASP.NET Core

Chapter 39: Onion Architecture

This chapter explains Onion Architecture, where the domain sits at the center and all dependencies point inward. You will learn how it compares with Clean Architecture and when it provides a clean structure for enterprise applications.

- What is Onion Architecture?
- Core domain at the center
- Dependency flow
- Onion vs Clean Architecture
- Benefits and drawbacks

Chapter 40: Domain-Driven Design (DDD) Architecture

This chapter introduces DDD as a way to model software based on real business concepts, not database tables. You will learn key building blocks like entities, value objects, aggregates, and bounded contexts, and also understand when DDD is overkill.

- What is DDD?
- Strategic vs Tactical DDD
- Core DDD concepts
  - Entities
  - Value Objects
  - Aggregates
  - Repositories
  - Domain Services
- Bounded Context
- DDD with Clean Architecture
- When DDD is overkill

Follow Us on Social Media

👥 **Join our Telegram Group:** https://telegram.me/dotnettutorials
📺 **Subscribe to our YouTube Channel:** https://www.youtube.com/@DotNetTutorials/playlists
👥 **Join Our Facebook Group:** https://www.facebook.com/groups/501905403704561
📘 **Like & Follow Our Facebook Page:** https://www.facebook.com/tutorialsdotnet
💼 **Join Our LinkedIn Group:** https://www.linkedin.com/groups/14539101/
🏢 **Follow Our LinkedIn Company Page:** https://www.linkedin.com/company/dot-net-tutorials/